Week 4 - Univariate Analysis, part 2

1. Lesson - None

2. Weekly graph question

Below are a histogram and boxplot representation of the same data. A pharmacy is keeping a record of the prices of the drugs that it sells, and an administrator wants to know how much the more expensive drugs tend to cost, in the context of the other prices.

Please write a short explanation of the pros and cons of these two representations. Which would you choose? How would you modify the formatting, if at all, to make it more visually interesting, clear, or informative?

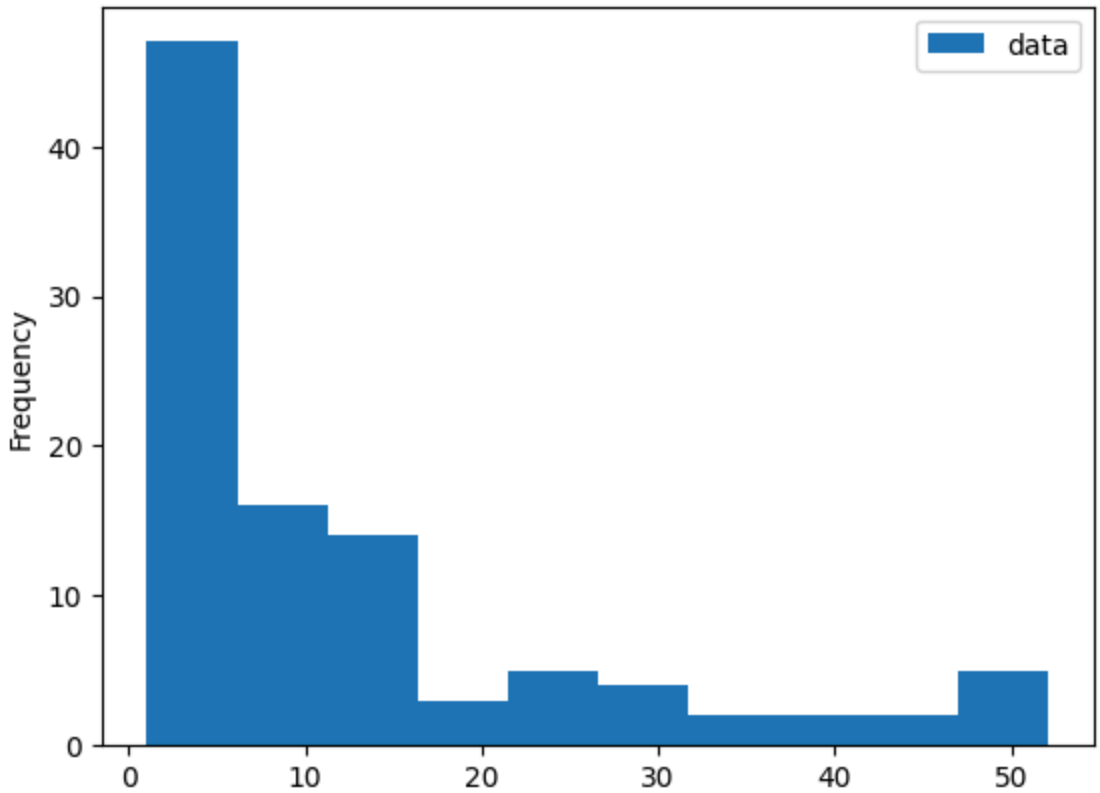
In [75]:
```
import numpy as np
import pandas as pd

np.random.seed(0)
num_data = 100
data = np.exp(np.random.uniform(size = num_data) * 4)
df = pd.DataFrame(data.T, columns = ["data"])
```

In [76]:
```
print("The 75th percentile is:", df.quantile(q = 0.75))
df.plot.hist()
```

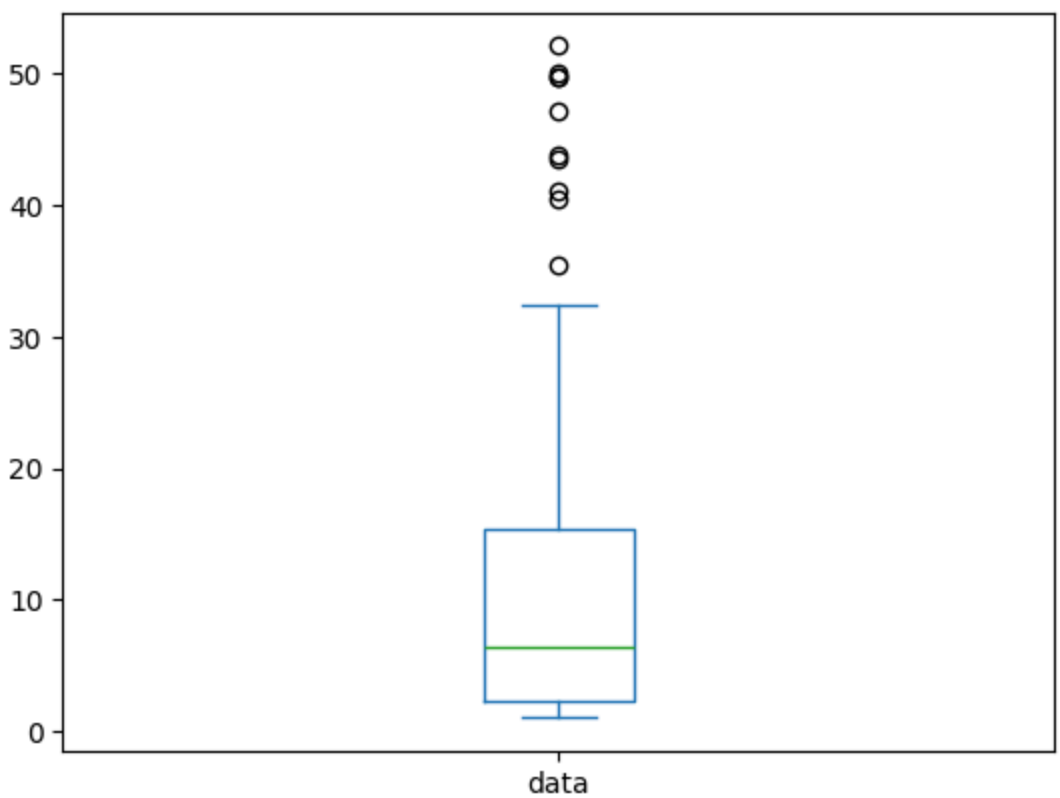
```
The 75th percentile is: data    15.457656
Name: 0.75, dtype: float64
```

Out[76]:
```
<Axes: ylabel='Frequency'>
```



In [77]:
```
df.plot.box()
```

Out[77]: <Axes: >



The histogram and box plot both make it possible to see the distribution of a feature

The histogram makes it easy to visualize the distribution. The reader can easily scan the bins to see which price ranges for the drug are most common, and what the low/high end prices are. The disadvantage of the histogram is you cannot easily identify metrics like the median or average price.

The boxplot also shows the distribution, but it additionally clearly displays what the outlier prices are. We can see the prices above 35 are outliers. It also shows the quartiles for the distributions, so we know the median is around 7, the upper quartile is around 15, and the lower quartile is around 3. It also shows the whiskers are around 2 and 35.

I would utilize the histogram for the administrator's inquiry. The histogram is easier to read for a business partner, and we can show given a the current price where it would fall on the distribution. I would add some additional detail like a title, X axis label, and if the user was interested in a metric like mean, we could add a line on the graph indicating where that falls.

3. Homework - working on your datasets

This week, you will do the same types of exercises as last week, but you should use your chosen datasets that someone in your class found last semester. (They likely will not be the particular datasets that you found yourself.)

Here are some types of analysis you can do:

- Draw histograms and histogram variants for each feature or column. (Swarm plot, kde plot, violin plot).
- Draw grouped histograms. For instance, if you have tree heights for both maple and oak trees, you could draw histograms for both.
- Draw a bar plot to indicate total counts of each categorical variable in a given column.
- Find means, medians, and modes.

Conclusions:

- Explain what conclusions you would draw from this analysis: are the data what you expect? Are the data likely to be usable? If they are not useable, find some new data!
- What is the overall shape of the distribution? Is it normal, skewed, bimodal, uniform, etc.?
- Are there any outliers present? (Data points that are far from the others.)
- If there are multiple related histograms, how does the distribution change across different groups?

- What are the minimum and maximum values represented in each histogram?
- How do bin sizes affect the histogram? Does changing the bin width reveal different patterns in the data?
- Does the distribution appear normal, or does it have a different distribution?

In [78]:
```
import matplotlib.pyplot as plt
```

Read Data

In [79]:
```
# https://www.kaggle.com/datasets/danb91/covid-ppp-loan-data-with-fraud-examples
df = pd.read_csv("C:/Users/caleb/Downloads/ppploan/ppp_over_150k.csv")
df
```

Out[79]:

	LoanNumber	DateApproved	SBAOfficeCode	ProcessingMethod	BorrowerName
0	9547507704	2020-05-01	464	PPP	sumter coatings inc
1	9777677704	2020-05-01	464	PPP	pleasant places inc
2	5791407702	2020-05-01	1013	PPP	boyer childrens clinic
3	6223567700	2020-05-01	920	PPP	kirtley construction inc
4	9662437702	2020-05-01	101	PPP	aero box llc
...	...	...	...	...	...
968527	4395967002	2020-04-03	897	PPP	roy e paulson jr pc
968528	6985647108	2020-04-14	897	PPP	sweetwater county child developmental center inc
968529	7996438405	2021-02-12	897	PPS	electrical systems of wyoming inc
968530	9054647103	2020-04-15	897	PPP	eden life care
968531	9184687004	2020-04-09	897	PPP	s s johnson enterprises inc

968532 rows × 66 columns

In [80]:
```
fraud_df = pd.read_csv("C:/Users/caleb/Downloads/ppploan/ppp_fraud_cases.csv")
fraud_df
```

Out[80]:

	Company	State	Source	Suspects
0	apollo hp inc	CA	https://www.justice.gov/usao-edca/pr/rocklin-b...	[['Kevin Lee Co', 'CA']]
1	power house sports academy	GA	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Micah K Bais...
2	faithful transport services llc	SC	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Travis C Cro...
3	kmj transport llc	SC	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Keith A Malo...
4	market yourself inc	GA	https://www.justice.gov/opa/pr/six-charged-con...	[['Rodericque Thompson', 'GA'], ['Tabronx W Sm...
...	...	...	...	...
95	pink lady line	TX	https://www.justice.gov/usao-sdtx/pr/local-wom...	[['LaDonna Wiggins', 'TX']]
96	guardian of humanity inc	AZ	https://www.justice.gov/usao-wdny/pr/north-chi...	[['Michael Rech', 'NY']]
97	skilled trade investments lp	TX	https://www.justice.gov/opa/pr/texas-man-arres...	[['Scott Jackson Davis', 'TX']]
98	skilled trade staffing llc	TX	https://www.justice.gov/opa/pr/texas-man-arres...	[['Scott Jackson Davis', 'TX']]
99	skilled trade investments gp llc	TX	https://www.justice.gov/opa/pr/texas-man-arres...	[['Scott Jackson Davis', 'TX']]

100 rows × 4 columns

In [81]:
```
# Label the fraudulent rows based on company names in the label dataset
df['Fraud'] = df['BorrowerName'].isin(fraud_df['Company']).astype(int)
```

In [82]:
```
# Sanity check the join
len(fraud_df['Company'].unique())
```

Out[82]: 100

In [83]:
```
# We only see 83 of the 100 fraudulent companies... there may be invalid company na
len(set(df[df['Fraud'] == 1]['BorrowerName'].unique()))
```

Out[83]: 83

In [84]:
```
not_found_fraudulent_companies = set(
    fraud_df['Company'].unique()
).difference(set(
    df[df['Fraud'] == 1]['BorrowerName'].unique()
))
not_found_fraudulent_companies
```

Out[84]:
```
{'aboveall business',
 'all star room and board services of michigan inc',
 'blue flight logistics llc',
 'dituri construction llc',
 'eni operation co',
 'eric r shibley md pllc',
 'es1 llc',
 'maturino enterprises',
 'nkb enterprise llc',
 'pinnacle industry inc',
 'seattles finest cannabis llc',
 'sfc llc',
 'showtime trucking xpo inc',
 'ss1 llc',
 'velocity innovations',
 'visla captial llc',
 'wingate funeral service group'}
```

In [85]:
```
# Transforming the company names to make the search less stringent. We may be able
companies = [
    'aboveall',
    'all star room',
    'blue flight',
    'dituri',
    'eni operation',
    'shibley',
    'es1',
    'maturino',
    'nkb enterprise',
    'pinnacle industry',
    'seattles finest cannabis',
    'sfc',
    'showtime trucking ',
    'ss1',
    'velocity',
    'visla captial ',
    'wingate funeral'
]
for c in companies:
    print(f"{c}: {df['BorrowerName'].str.contains(c).any()}")
    print(df[df['BorrowerName'].str.contains(c, na=False)]['BorrowerName'])
    print('\n')
```

```
aboveall: False
Series([], Name: BorrowerName, dtype: object)


all star room: True
454316    all star room  board services of michigan inc
Name: BorrowerName, dtype: object


blue flight: False
Series([], Name: BorrowerName, dtype: object)


dituri: True
588733    drs gardiner  dituri pasba small 7a term
Name: BorrowerName, dtype: object


eni operation: False
Series([], Name: BorrowerName, dtype: object)


shibley: True
371855                  shibleyville llc
373694                  shibleyville llc
623978          shibley operating co llc
623979          shibley operating co llc
680901            shibley management inc
684010            shibley management inc
692598    spangenberg shibley  liber llp
692680    spangenberg shibley  liber llp
Name: BorrowerName, dtype: object


es1: True
25546     connie a hottmann dba gampg enterprises1
85495                                pawsibilities1
107014                          pawsibilities1 inc
697585                   gc aviation services1 llc
962841                                 naples15llc
Name: BorrowerName, dtype: object


maturino: False
Series([], Name: BorrowerName, dtype: object)


nkb enterprise: False
Series([], Name: BorrowerName, dtype: object)


pinnacle industry: True
717516    pinnacle industry llc
Name: BorrowerName, dtype: object
```

```
seattles finest cannabis: False
Series([], Name: BorrowerName, dtype: object)


sfc: True
48382                      sfc communications inc
142657                                  dasfc inc
144540                                   sfcj inc
155501                          sfcs van ness llc
157550                          sfcs van ness llc
270256        sfc contract services of georgia llc
270257        sfc contract services of georgia llc
385224                   sfc contract services inc
559203             sfc engineering partnership inc
646209                                   rsfc llc
679289                                   rsfc llc
697190                                    sfc ltd
697322                                    sfc ltd
766923                                   lsfc inc
768870                   sfc delivery services inc
777362    sfc capillas exequiales y funerarias llc
845500                                   qsfc ltd
849726                         sfc management llc
855813                                   qsfc llc
879229                                   sfcc inc
896933                                   sfcs inc
Name: BorrowerName, dtype: object


showtime trucking : True
572055    showtime trucking llc
572361    showtime trucking llc
Name: BorrowerName, dtype: object


ss1: True
98072     rss1 walnut creek llc
137829    rss1 walnut creek llc
218370     fitness1 kendall llc
243950    mattress1holding corp
706107                hrss1 llc
708289                hrss1 llc
Name: BorrowerName, dtype: object


velocity: True
2806                 velocity ventures inc
25422                    velocityscape llc
41744     infinite velocity automotive inc
41745     infinite velocity automotive inc
43018                  velocity motors llc
                         ...
929025       velocity consulting solutions
929415       velocity consulting solutions
947116                velocity machine inc
951280              velocity transport inc
```

```
959232            velocity water works llc
Name: BorrowerName, Length: 106, dtype: object


visla captial : False
Series([], Name: BorrowerName, dtype: object)


wingate funeral: False
Series([], Name: BorrowerName, dtype: object)
```

Summarize Data

In [86]:
```
df.describe()
```

```
c:\Users\caleb\anaconda3\envs\iris-fisher\lib\site-packages\pandas\core\nanops.py:10
16: RuntimeWarning: invalid value encountered in subtract
  sqr = _ensure_numeric((avg - values) ** 2)
```

Out[86]:

	LoanNumber	SBAOfficeCode	Term	SBAGuarantyPercentage	InitialApprov
count	9.685320e+05	968532.000000	968532.000000	968532.0	9.68
mean	5.427133e+09	571.520066	36.297555	100.0	5.32
std	2.551320e+09	263.025314	17.214728	0.0	7.44
min	1.000007e+09	101.000000	0.000000	100.0	0.00
25%	3.271096e+09	373.000000	24.000000	100.0	2.00
50%	5.400677e+09	515.000000	24.000000	100.0	2.95
75%	7.546310e+09	811.000000	60.000000	100.0	5.40
max	9.999007e+09	1094.000000	120.000000	100.0	1.00

8 rows × 33 columns

In [87]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 968532 entries, 0 to 968531
Data columns (total 67 columns):
 #   Column                        Non-Null Count   Dtype
---  ------                        --------------   -----
 0   LoanNumber                    968532 non-null  int64
 1   DateApproved                  968532 non-null  object
 2   SBAOfficeCode                 968532 non-null  int64
 3   ProcessingMethod              968532 non-null  object
 4   BorrowerName                  968525 non-null  object
 5   BorrowerAddress               968532 non-null  object
 6   BorrowerCity                  968532 non-null  object
 7   BorrowerState                 968532 non-null  object
 8   BorrowerZip                   968532 non-null  object
 9   LoanStatusDate                968532 non-null  object
 10  LoanStatus                    968532 non-null  object
 11  Term                          968532 non-null  int64
 12  SBAGuarantyPercentage         968532 non-null  int64
 13  InitialApprovalAmount         968532 non-null  float64
 14  CurrentApprovalAmount         968532 non-null  float64
 15  UndisbursedAmount             968532 non-null  float64
 16  FranchiseName                 968532 non-null  object
 17  ServicingLenderLocationID     968532 non-null  int64
 18  ServicingLenderName           968532 non-null  object
 19  ServicingLenderAddress        968532 non-null  object
 20  ServicingLenderCity           968532 non-null  object
 21  ServicingLenderState          968532 non-null  object
 22  ServicingLenderZip            968532 non-null  object
 23  RuralUrbanIndicator           968532 non-null  object
 24  HubzoneIndicator              968532 non-null  object
 25  LMIIndicator                  968532 non-null  object
 26  BusinessAgeDescription        968532 non-null  object
 27  ProjectCity                   968532 non-null  object
 28  ProjectCountyName             968532 non-null  object
 29  ProjectState                  968532 non-null  object
 30  ProjectZip                    968532 non-null  object
 31  CD                            968532 non-null  object
 32  JobsReported                  968532 non-null  float64
 33  NAICSCode                     968532 non-null  int64
 34  Race                          968532 non-null  object
 35  Ethnicity                     968532 non-null  object
 36  UTILITIES_PROCEED             968532 non-null  float64
 37  PAYROLL_PROCEED               968532 non-null  float64
 38  MORTGAGE_INTEREST_PROCEED     968532 non-null  float64
 39  RENT_PROCEED                  968532 non-null  float64
 40  REFINANCE_EIDL_PROCEED        968532 non-null  float64
 41  HEALTH_CARE_PROCEED           968532 non-null  float64
 42  DEBT_INTEREST_PROCEED         968532 non-null  float64
 43  BusinessType                  968532 non-null  object
 44  OriginatingLenderLocationID   968532 non-null  int64
 45  OriginatingLender             968532 non-null  object
 46  OriginatingLenderCity         968532 non-null  object
 47  OriginatingLenderState        968532 non-null  object
 48  Gender                        968532 non-null  object
 49  Veteran                       968532 non-null  object
 50  NonProfit                     968532 non-null  object
```

```
 51  ForgivenessAmount              968532 non-null  float64
 52  ForgivenessDate                968532 non-null  object
 53  ApprovalDiff                   968532 non-null  float64
 54  NotForgivenAmount              968532 non-null  float64
 55  ForgivenPercentage             968532 non-null  float64
 56  TOTAL_PROCEED                  968532 non-null  float64
 57  PROCEED_Diff                   968532 non-null  float64
 58  UTILITIES_PROCEED_pct          968532 non-null  float64
 59  PAYROLL_PROCEED_pct            968532 non-null  float64
 60  MORTGAGE_INTEREST_PROCEED_pct  968532 non-null  float64
 61  RENT_PROCEED_pct               968532 non-null  float64
 62  REFINANCE_EIDL_PROCEED_pct     968532 non-null  float64
 63  HEALTH_CARE_PROCEED_pct        968532 non-null  float64
 64  DEBT_INTEREST_PROCEED_pct      968532 non-null  float64
 65  PROCEED_Per_Job                968532 non-null  float64
 66  Fraud                          968532 non-null  int64
dtypes: float64(25), int64(8), object(34)
memory usage: 495.1+ MB
```

There are no missing values

Check for Infinite Numeric Values and Duplicates

In [88]:
```
numeric_df = df.select_dtypes(include=[np.number])

# Find rows where ANY column has an infinite value
rows_with_inf = df[np.isinf(numeric_df).any(axis=1)]
rows_with_inf
```

Out[88]:

	LoanNumber	DateApproved	SBAOfficeCode	ProcessingMethod	BorrowerName
123812	6557167306	2020-04-30	914	PPP	st margaret mary school
550932	5780187005	2020-04-06	766	PPP	reliant transportation inc
763783	4864227203	2020-04-27	303	PPP	vincera rehab llc
967206	4563247009	2020-04-03	897	PPP	weeden construction llc

4 rows × 67 columns

In [89]:
```
# We will replaced these inf values with NaN
df.replace([np.inf, -np.inf], np.nan, inplace=True)
```

In [90]:
```
# There are no duplicates
duplicates = df[df.duplicated(keep=False)]
duplicates
```

Out[90]:

LoanNumber	DateApproved	SBAOfficeCode	ProcessingMethod	BorrowerName	Borrow

0 rows × 67 columns

Check for Inconsistent Data

In [91]:

```
# Validate amounts
amount_cols = [c for c in df.columns if "Amount" in c]
for a in amount_cols:
    print(f"Column: {a}")
    print(f"Min Val: {df[a].min()}, Max Val: {df[a].max()}")
    print(f"Number of occurences of Min Val: {len(df[df[a] == df[a].min()])}, numbe
    print("\n")
```

```
Column: InitialApprovalAmount
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 2, number of occurences of Max Val: 747


Column: CurrentApprovalAmount
Min Val: 150000.0, Max Val: 10000000.0
Number of occurences of Min Val: 6690, number of occurences of Max Val: 707


Column: UndisbursedAmount
Min Val: 0.0, Max Val: 3397100.0
Number of occurences of Min Val: 968515, number of occurences of Max Val: 1


Column: ForgivenessAmount
Min Val: 0.0, Max Val: 10196272.22
Number of occurences of Min Val: 93710, number of occurences of Max Val: 1


Column: NotForgivenAmount
Min Val: -196272.22000000067, Max Val: 10000000.0
Number of occurences of Min Val: 1, number of occurences of Max Val: 118
```

For InitialApprovalAmount, CurrentApprovalAmount, and NotForgivenAmount, the maximum value of 10000000.0 being common makes sense since the maximum allowed amount was $10,000,000 for an individual borrower (https://www.jw.com/news/insights-2021-paycheck-protection-program-loans/)

For CurrentApprovalAmount, the 150,000 minimum looks to be clipped. It is likely 150k loans or lower

The rest of the amounts do not seem to be inconsistent

In [92]:
```
# Validate proceeds
proceed_cols = [c for c in df.columns if "PROCEED" in c]
for p in proceed_cols:
    print(f"Column: {p}")
    print(f"Min Val: {df[p].min()}, Max Val: {df[p].max()}")
    print(f"Number of occurences of Min Val: {len(df[df[p] == df[p].min()])}, numbe
    print("\n")
```

```
Column: UTILITIES_PROCEED
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 635755, number of occurences of Max Val: 1


Column: PAYROLL_PROCEED
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 1931, number of occurences of Max Val: 512


Column: MORTGAGE_INTEREST_PROCEED
Min Val: 0.0, Max Val: 10000000.0
Number of occurences of Min Val: 939898, number of occurences of Max Val: 1


Column: RENT_PROCEED
Min Val: 0.0, Max Val: 5518277.93
Number of occurences of Min Val: 875030, number of occurences of Max Val: 1


Column: REFINANCE_EIDL_PROCEED
Min Val: 0.0, Max Val: 2951590.0
Number of occurences of Min Val: 966784, number of occurences of Max Val: 1


Column: HEALTH_CARE_PROCEED
Min Val: 0.0, Max Val: 3880000.0
Number of occurences of Min Val: 916822, number of occurences of Max Val: 1


Column: DEBT_INTEREST_PROCEED
Min Val: 0.0, Max Val: 2497617.0
Number of occurences of Min Val: 952173, number of occurences of Max Val: 1


Column: TOTAL_PROCEED
Min Val: 94800.0, Max Val: 10000000.0
Number of occurences of Min Val: 1, number of occurences of Max Val: 639


Column: PROCEED_Diff
Min Val: -14997.0, Max Val: 323707.0
Number of occurences of Min Val: 1, number of occurences of Max Val: 1


Column: UTILITIES_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 860335, number of occurences of Max Val: 1014


Column: PAYROLL_PROCEED_pct
Min Val: 0.0, Max Val: 1.041
Number of occurences of Min Val: 1935, number of occurences of Max Val: 1


Column: MORTGAGE_INTEREST_PROCEED_pct
```

```
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 940120, number of occurences of Max Val: 213


Column: RENT_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 875213, number of occurences of Max Val: 76


Column: REFINANCE_EIDL_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 966847, number of occurences of Max Val: 122


Column: HEALTH_CARE_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 916883, number of occurences of Max Val: 41


Column: DEBT_INTEREST_PROCEED_pct
Min Val: 0.0, Max Val: 1.0
Number of occurences of Min Val: 952318, number of occurences of Max Val: 5


Column: PROCEED_Per_Job
Min Val: 0.0, Max Val: 6512997.5
Number of occurences of Min Val: 1931, number of occurences of Max Val: 1
```

The proceed columns tell a similar story. Some max out at $10,000,000. The minimums are mostly 0$. No inconsistent data looks to be present

Feature Observations

The following numeric columns should not be used for modeling:

- Loan Number (ID)
- SBAOfficeCode (ID, though maybe it matters if certain locations are approving all fraudulent loans)
- SBAGuarantyPercentage(Always value 100)
- UndisbursedAmount (Always value 0)
- ServicingLenderLocationID (ID)
- NAICSCode (ID)
- OriginatingLenderLocationID (ID)

The following column have numeric dtype but are categorical

- Term

The following non-numeric columns should not be used for modeling:

- BorrowerAddress (Random string)
- BorrowerName (Random string)
- Ethnicity (Bias)
- FranchiseName (random str)
- Gender (Bias)
- OriginatingLender (Random str)
- Race (bias)
- ServicingLenderAddress (random str)
- ServicingLenderName (random str)

There are also a few non-numeric columns that are categorical but have many values. I will skip them for initial analysis but may revisit them if they end up being useful

- BorrowerCity (categorical but not specific enough to be useful)
- BorrowerZip (categorical but not specific enough to be useful)
- CD (categorical but not specific enough to be useful)
- OriginatingLenderCity (categorical but not specific enough to be useful)
- ProjectCity (categorical but not specific enough to be useful)
- ProjectCountyName (categorical but not specific enough to be useful)
- ProjectZip (categorical but not specific enough to be useful)
- ServicingLenderCity (categorical but not specific enough to be useful)
- ServicingLenderZip (categorical but not specific enough to be useful)

Non numeric cols which can be used for modeling (one hot/ordinal encoding or datetime to numeric conversion)

- BusinessType (categorical)
- BusinessAgeDescription (categorical)
- DateApproved (datetime, maybe convert to int)
- ForgivenessDate (datetime, maybe convert to int)
- HubzoneIndicator (categorical)
- LMIndicator (categorical)
- LoanStatus (categorical)
- LoanStatusDate (datetime, maybe convert to int)
- NonProfit (categorical)
- ProcessingMethod (categorical)
- RuralUrbanIndicator (categorical)
- Veteran (categorical)

In [93]:
```
numeric_cols_to_drop = [
    'LoanNumber',
    'SBAOfficeCode',
    'SBAGuarantyPercentage',
    'UndisbursedAmount',
    'ServicingLenderLocationID',
```

```
    'NAICSCode',
    'OriginatingLenderLocationID',
]
```

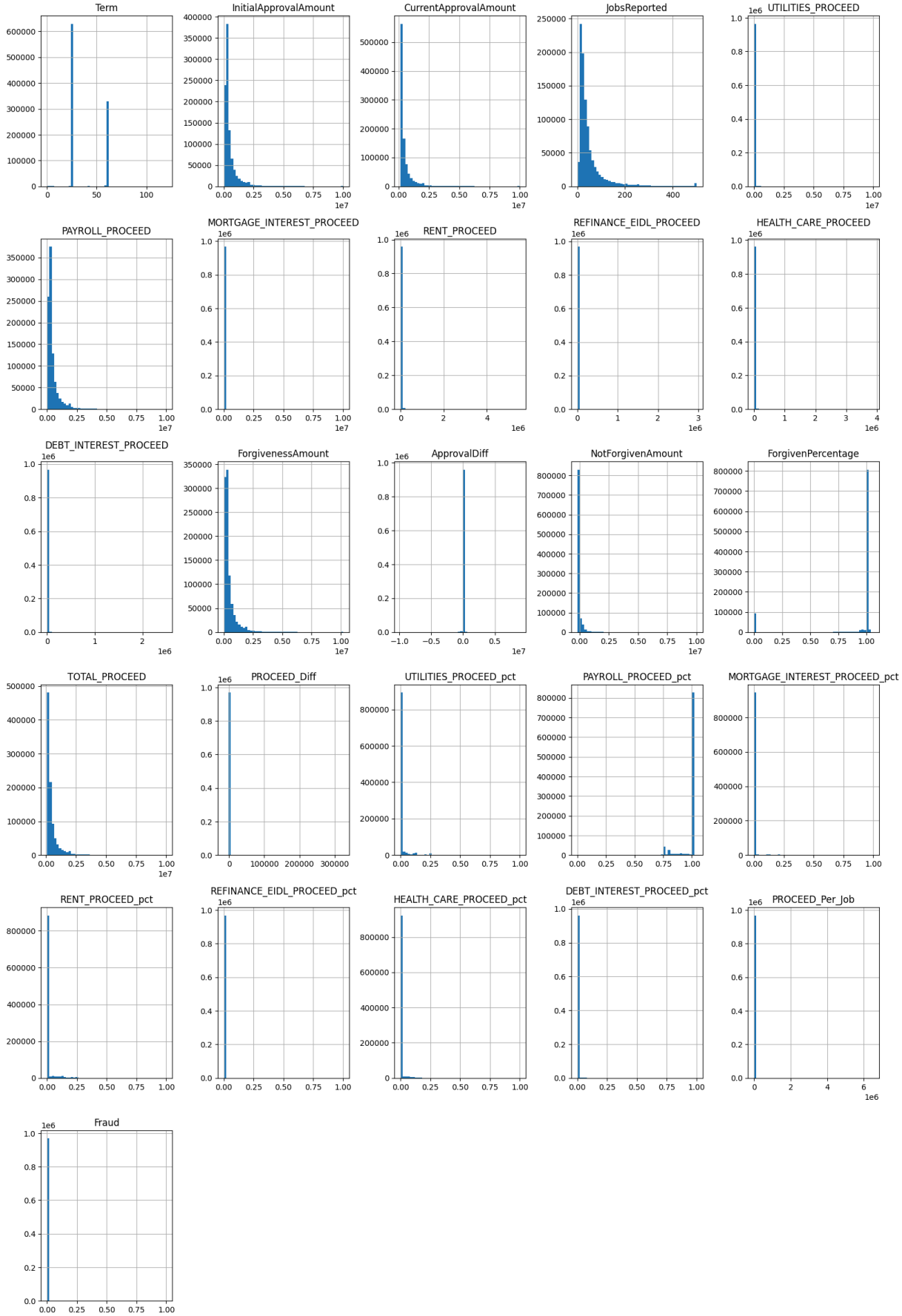
In [94]:
```
non_numeric_cols_to_drop = [
    'BorrowerCity',
    'BorrowerZip',
    'CD',
    'OriginatingLenderCity',
    'ProjectCity',
    'ProjectCountyName',
    'ProjectZip',
    'ServicingLenderCity',
    'ServicingLenderZip',
    'BorrowerAddress',
    'BorrowerName',
    'Ethnicity',
    'FranchiseName',
    'Gender',
    'OriginatingLender',
    'Race',
    'ServicingLenderAddress',
    'ServicingLenderName',
]
```

In [95]:
```
columns_to_drop = set(numeric_cols_to_drop).union(set(non_numeric_cols_to_drop))
df = df.drop(columns_to_drop, axis=1)
```

In [96]:
```
numeric_cols = df.select_dtypes(include='number').columns
non_numeric_cols = df.columns.difference(numeric_cols)
```

Histograms

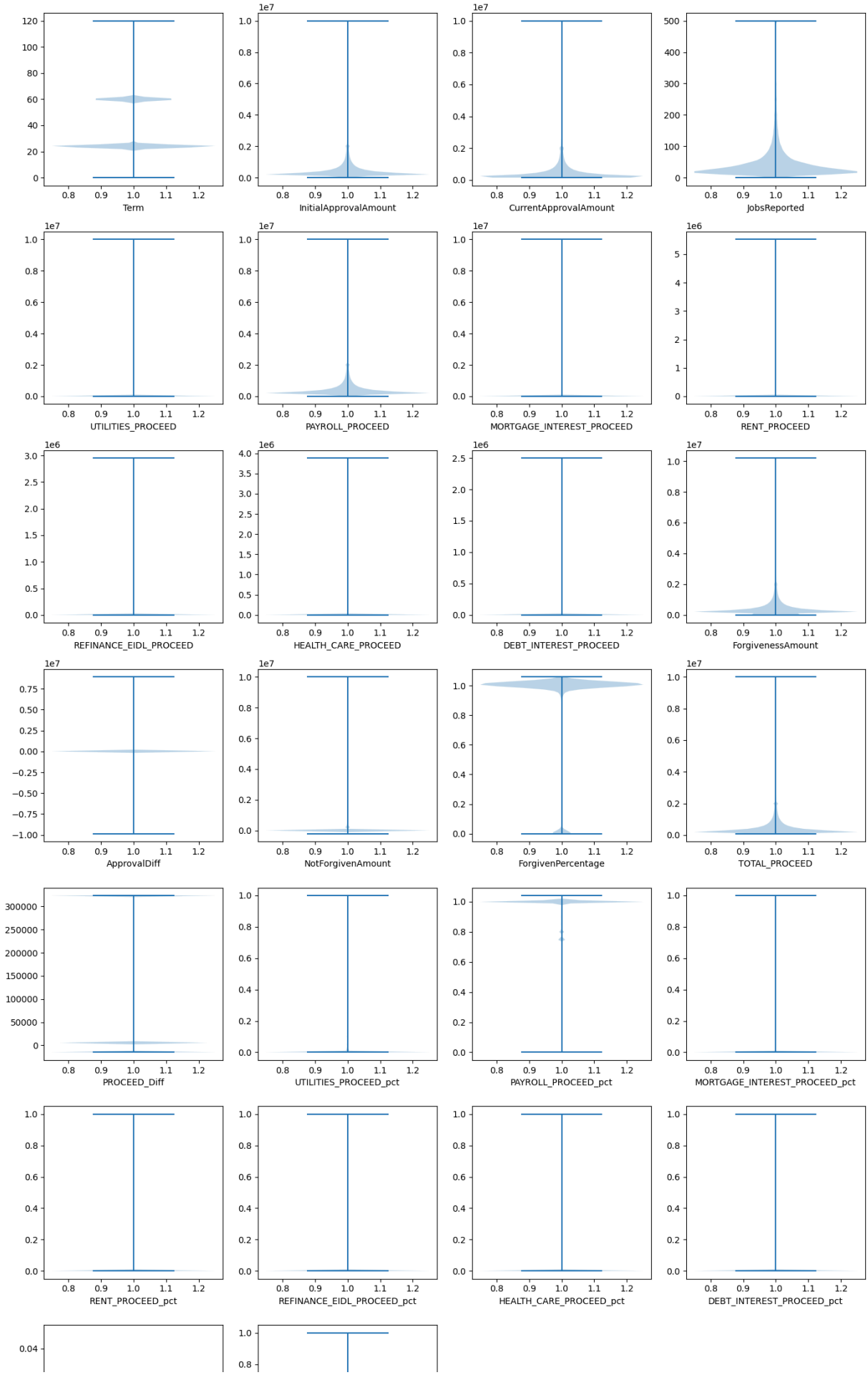
In [97]:
```
df[numeric_cols].hist(column=numeric_cols, figsize=(16, 24), bins=50)
plt.tight_layout()
None
```

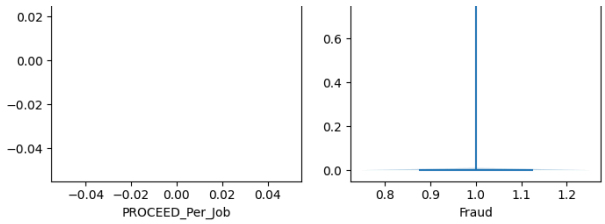


Violin Plots

In [98]:
```
num_cols = 4
num_rows = (len(numeric_cols) + num_cols - 1) // num_cols
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))
for (i,c) in enumerate(numeric_cols):
    plt.subplot(num_rows, num_cols, i+1)
    plt.violinplot(df[c])
    plt.xlabel(c)
plt.tight_layout()
None
```

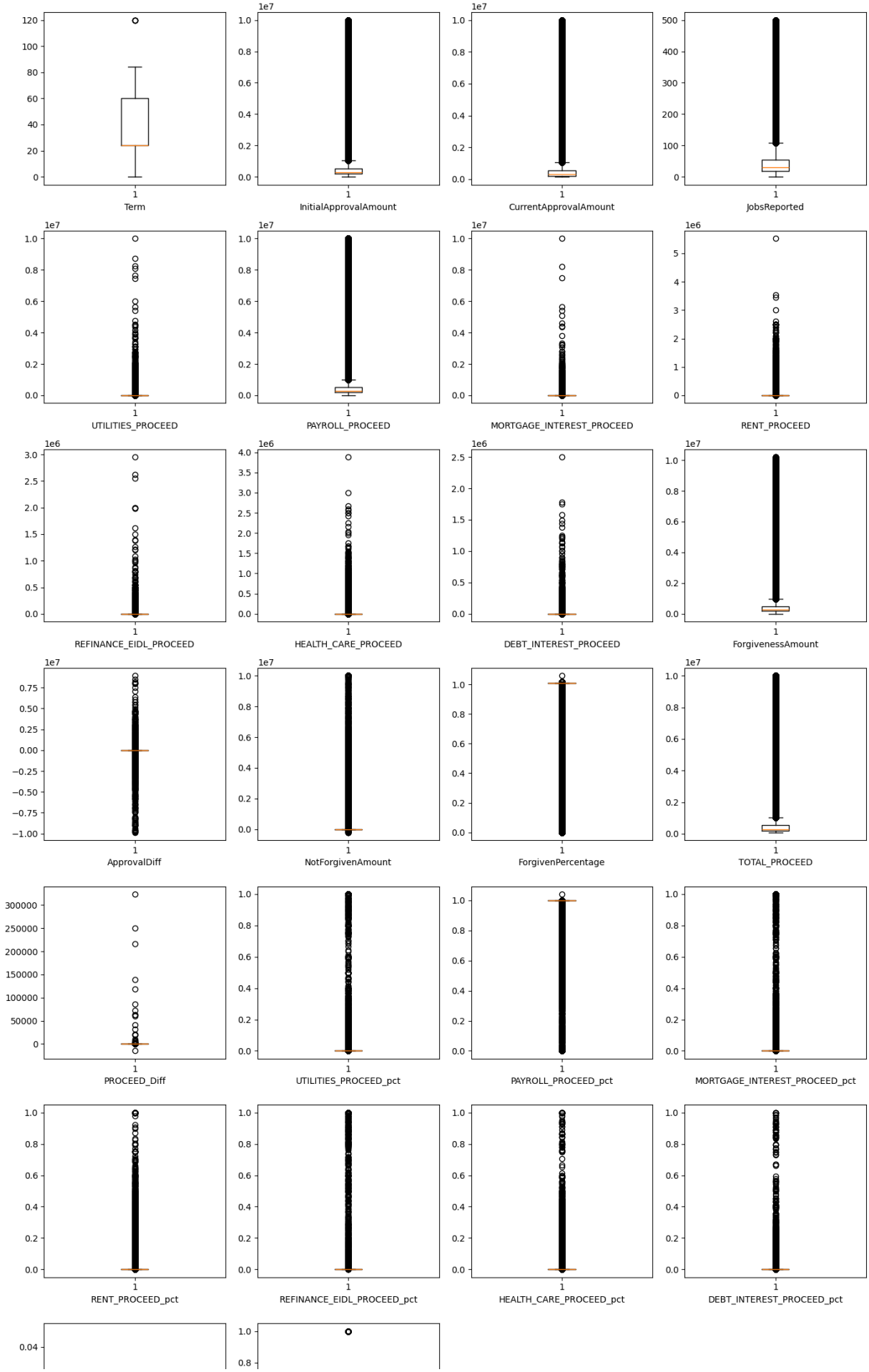
```
c:\Users\caleb\anaconda3\envs\iris-fisher\lib\site-packages\numpy\linalg\_linalg.py:
2383: RuntimeWarning: invalid value encountered in det
  r = _umath_linalg.det(a, signature=signature)
```

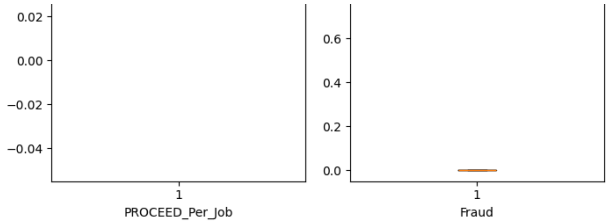

Term
InitialApprovalAmount
CurrentApprovalAmount
JobsReported
UTILITIES_PROCEED
PAYROLL_PROCEED
MORTGAGE_INTEREST_PROCEED
RENT_PROCEED
REFINANCE_EIDL_PROCEED
HEALTH_CARE_PROCEED
DEBT_INTEREST_PROCEED
ForgivenessAmount
ApprovalDiff
NotForgivenAmount
ForgivenPercentage
TOTAL_PROCEED
PROCEED_Diff
UTILITIES_PROCEED_pct
PAYROLL_PROCEED_pct
MORTGAGE_INTEREST_PROCEED_pct
RENT_PROCEED_pct
REFINANCE_EIDL_PROCEED_pct
HEALTH_CARE_PROCEED_pct
DEBT_INTEREST_PROCEED_pct



Boxplots

In [99]:
```
num_cols = 4
num_rows = (len(numeric_cols) + num_cols - 1) // num_cols
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))
for (i,c) in enumerate(numeric_cols):
    plt.subplot(num_rows, num_cols, i+1)
    plt.boxplot(df[c])
    plt.xlabel(c)
plt.tight_layout()
None
```


Term
InitialApprovalAmount
CurrentApprovalAmount
JobsReported
UTILITIES_PROCEED
PAYROLL_PROCEED
MORTGAGE_INTEREST_PROCEED
RENT_PROCEED
REFINANCE_EIDL_PROCEED
HEALTH_CARE_PROCEED
DEBT_INTEREST_PROCEED
ForgivenessAmount
ApprovalDiff
NotForgivenAmount
ForgivenPercentage
TOTAL_PROCEED
PROCEED_Diff
UTILITIES_PROCEED_pct
PAYROLL_PROCEED_pct
MORTGAGE_INTEREST_PROCEED_pct
RENT_PROCEED_pct
REFINANCE_EIDL_PROCEED_pct
HEALTH_CARE_PROCEED_pct
DEBT_INTEREST_PROCEED_pct



Plotting Categorical Columns

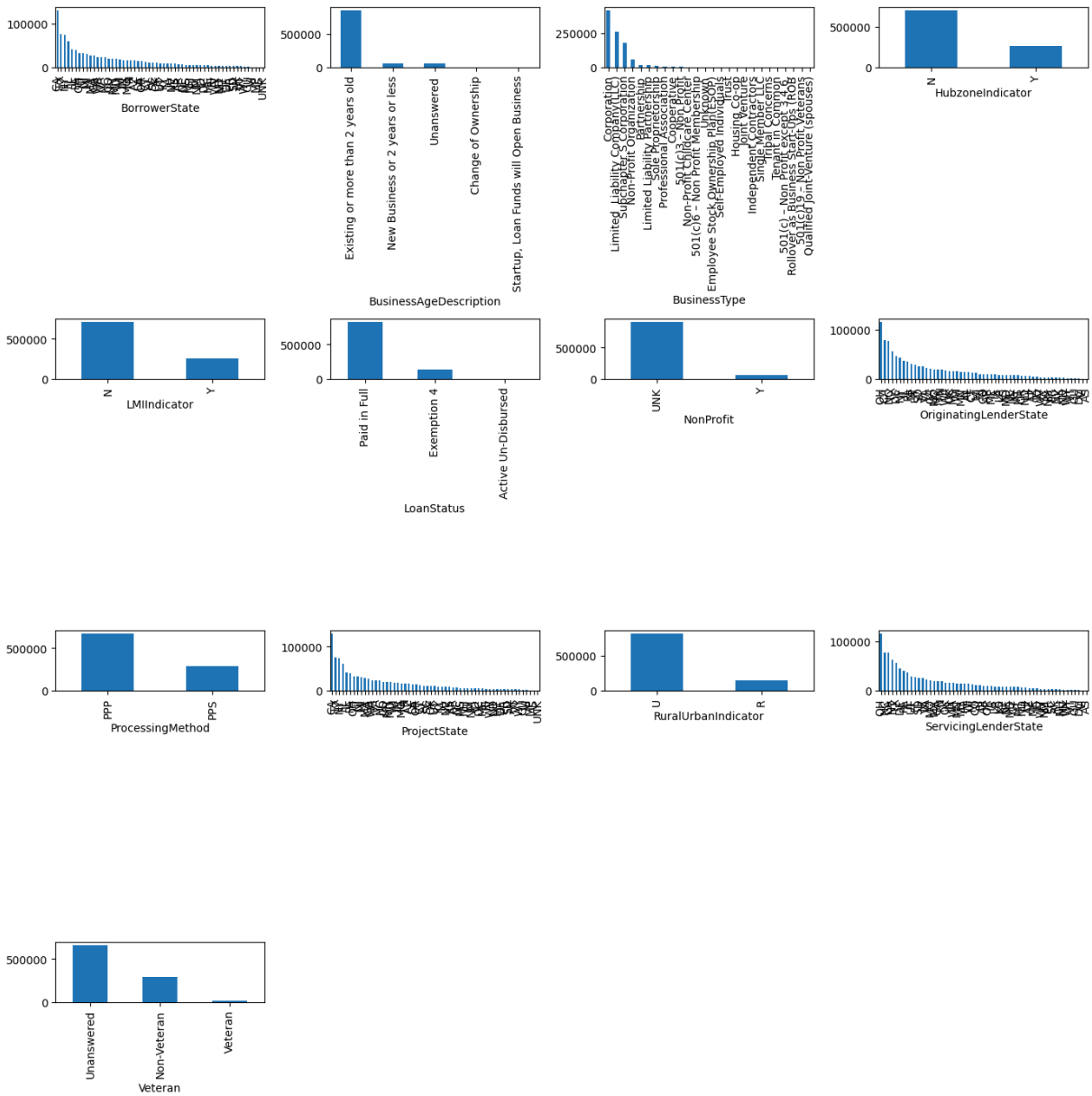
In [100…
```
categorical_cols = [c for c in non_numeric_cols if df[c].nunique() <= 100]
```

In [101…
```
set(non_numeric_cols) - set(categorical_cols)
```

Out[101…
```
{'DateApproved', 'ForgivenessDate', 'LoanStatusDate'}
```

In [104…
```
date_columns = [
    'DateApproved',
    'ForgivenessDate',
    'LoanStatusDate',
]
```

In [102…
```
num_cols = 4
num_rows = (len(categorical_cols) + num_cols - 1) // num_cols
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))
for (i,c) in enumerate(categorical_cols):
    plt.subplot(num_rows, num_cols, i+1)
    df[c].value_counts().plot(kind='bar')
    plt.xlabel(c)
plt.tight_layout()
None
```



Encoding Date Features

In [107…

```
for c in date_columns:
    print(df[c].head(5))
    print()
```

```
0    2020-05-01
1    2020-05-01
2    2020-05-01
3    2020-05-01
4    2020-05-01
Name: DateApproved, dtype: object

0    2020-11-20
1    2021-08-12
2    2021-02-10
3    2021-09-10
4    2021-04-08
Name: ForgivenessDate, dtype: object

0    2020-12-18
1    2021-09-28
2    2021-03-17
3    2021-10-16
4    2021-08-17
Name: LoanStatusDate, dtype: object
```

In [109…

```
for c in date_columns:
    df[f"{c}_datetime"] = pd.to_datetime(df[c], format='%Y-%m-%d')
    print(df[f"{c}_datetime"].head(5))
    df[f"{c}_int"] = df[f"{c}_datetime"].astype(np.int64) // 10**9
    print(df[f"{c}_int"].head(5))
    df.drop([c, f"{c}_datetime"], axis=1, inplace=True)
```

```
0   2020-05-01
1   2020-05-01
2   2020-05-01
3   2020-05-01
4   2020-05-01
Name: DateApproved_datetime, dtype: datetime64[ns]
0    1588291200
1    1588291200
2    1588291200
3    1588291200
4    1588291200
Name: DateApproved_int, dtype: int64
0   2020-11-20
1   2021-08-12
2   2021-02-10
3   2021-09-10
4   2021-04-08
Name: ForgivenessDate_datetime, dtype: datetime64[ns]
0    1605830400
1    1628726400
2    1612915200
3    1631232000
4    1617840000
Name: ForgivenessDate_int, dtype: int64
0   2020-12-18
1   2021-09-28
2   2021-03-17
3   2021-10-16
4   2021-08-17
Name: LoanStatusDate_datetime, dtype: datetime64[ns]
0    1608249600
1    1632787200
2    1615939200
3    1634342400
4    1629158400
Name: LoanStatusDate_int, dtype: int64
```

Encoding Categorical Features

In [114…

```
categorical_cols
```

Out[114…

```
['BorrowerState',
 'BusinessAgeDescription',
 'BusinessType',
 'HubzoneIndicator',
 'LMIIndicator',
 'LoanStatus',
 'NonProfit',
 'OriginatingLenderState',
 'ProcessingMethod',
 'ProjectState',
 'RuralUrbanIndicator',
 'ServicingLenderState',
 'Veteran']
```

I am going to Ordinal Encode each of these due to the size of the data. We can later attempt One-Hot encoding.

In [115…

```
from sklearn.preprocessing import OrdinalEncoder

enc = OrdinalEncoder()
enc.fit(df[categorical_cols])
enc.categories_
```

Out[115…

```
[array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MP', 'MS', 'MT', 'NC', 'ND', 'NE',
        'NH', 'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI',
        'SC', 'SD', 'TN', 'TX', 'UNK', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI',
        'WV', 'WY'], dtype=object),
 array(['Change of Ownership', 'Existing or more than 2 years old',
        'New Business or 2 years or less',
        'Startup, Loan Funds will Open Business', 'Unanswered'],
       dtype=object),
 array(['501(c) – Non Profit except 3,4,6,',
        '501(c)19 – Non Profit Veterans', '501(c)3 – Non Profit',
        '501(c)6 – Non Profit Membership', 'Cooperative', 'Corporation',
        'Employee Stock Ownership Plan(ESOP)', 'Housing Co-op',
        'Independent Contractors', 'Joint Venture',
        'Limited  Liability Company(LLC)', 'Limited Liability Partnership',
        'Non-Profit Childcare Center', 'Non-Profit Organization',
        'Partnership', 'Professional Association',
        'Qualified Joint-Venture (spouses)',
        'Rollover as Business Start-Ups (ROB', 'Self-Employed Individuals',
        'Single Member LLC', 'Sole Proprietorship',
        'Subchapter S Corporation', 'Tenant in Common', 'Tribal Concerns',
        'Trust', 'Unknown'], dtype=object),
 array(['N', 'Y'], dtype=object),
 array(['N', 'Y'], dtype=object),
 array(['Active Un-Disbursed', 'Exemption 4', 'Paid in Full'], dtype=object),
 array(['UNK', 'Y'], dtype=object),
 array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MS', 'MT', 'NC', 'ND', 'NE', 'NH',
        'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI', 'SC',
        'SD', 'TN', 'TX', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI', 'WV', 'WY'],
       dtype=object),
 array(['PPP', 'PPS'], dtype=object),
 array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MP', 'MS', 'MT', 'NC', 'ND', 'NE',
        'NH', 'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI',
        'SC', 'SD', 'TN', 'TX', 'UNK', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI',
        'WV', 'WY'], dtype=object),
 array(['R', 'U'], dtype=object),
 array(['AK', 'AL', 'AR', 'AS', 'AZ', 'CA', 'CO', 'CT', 'DC', 'DE', 'FL',
        'GA', 'GU', 'HI', 'IA', 'ID', 'IL', 'IN', 'KS', 'KY', 'LA', 'MA',
        'MD', 'ME', 'MI', 'MN', 'MO', 'MS', 'MT', 'NC', 'ND', 'NE', 'NH',
        'NJ', 'NM', 'NV', 'NY', 'OH', 'OK', 'OR', 'PA', 'PR', 'RI', 'SC',
        'SD', 'TN', 'TX', 'UT', 'VA', 'VI', 'VT', 'WA', 'WI', 'WV', 'WY'],
       dtype=object),
 array(['Non-Veteran', 'Unanswered', 'Veteran'], dtype=object)]
```

In [116…

```
df[categorical_cols] = enc.transform(df[categorical_cols])
```

In [118…

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 968532 entries, 0 to 968531
Data columns (total 42 columns):
 #   Column                          Non-Null Count   Dtype
---  ------                          --------------   -----
 0   ProcessingMethod                968532 non-null  float64
 1   BorrowerState                   968532 non-null  float64
 2   LoanStatus                      968532 non-null  float64
 3   Term                            968532 non-null  int64
 4   InitialApprovalAmount           968532 non-null  float64
 5   CurrentApprovalAmount           968532 non-null  float64
 6   ServicingLenderState            968532 non-null  float64
 7   RuralUrbanIndicator             968532 non-null  float64
 8   HubzoneIndicator                968532 non-null  float64
 9   LMIIndicator                    968532 non-null  float64
 10  BusinessAgeDescription          968532 non-null  float64
 11  ProjectState                    968532 non-null  float64
 12  JobsReported                    968532 non-null  float64
 13  UTILITIES_PROCEED               968532 non-null  float64
 14  PAYROLL_PROCEED                 968532 non-null  float64
 15  MORTGAGE_INTEREST_PROCEED       968532 non-null  float64
 16  RENT_PROCEED                    968532 non-null  float64
 17  REFINANCE_EIDL_PROCEED          968532 non-null  float64
 18  HEALTH_CARE_PROCEED             968532 non-null  float64
 19  DEBT_INTEREST_PROCEED           968532 non-null  float64
 20  BusinessType                    968532 non-null  float64
 21  OriginatingLenderState          968532 non-null  float64
 22  Veteran                         968532 non-null  float64
 23  NonProfit                       968532 non-null  float64
 24  ForgivenessAmount               968532 non-null  float64
 25  ApprovalDiff                    968532 non-null  float64
 26  NotForgivenAmount               968532 non-null  float64
 27  ForgivenPercentage              968532 non-null  float64
 28  TOTAL_PROCEED                   968532 non-null  float64
 29  PROCEED_Diff                    968532 non-null  float64
 30  UTILITIES_PROCEED_pct           968532 non-null  float64
 31  PAYROLL_PROCEED_pct             968532 non-null  float64
 32  MORTGAGE_INTEREST_PROCEED_pct   968532 non-null  float64
 33  RENT_PROCEED_pct                968532 non-null  float64
 34  REFINANCE_EIDL_PROCEED_pct      968532 non-null  float64
 35  HEALTH_CARE_PROCEED_pct         968532 non-null  float64
 36  DEBT_INTEREST_PROCEED_pct       968532 non-null  float64
 37  PROCEED_Per_Job                 968528 non-null  float64
 38  Fraud                           968532 non-null  int64
 39  DateApproved_int                968532 non-null  int64
 40  ForgivenessDate_int             968532 non-null  int64
 41  LoanStatusDate_int              968532 non-null  int64
dtypes: float64(37), int64(5)
memory usage: 310.4 MB
```

Conclusions

The dataset has a large number of instances (over 900,000) and features (66). The features are a mix of categorical and numeric. There are also some string features such as names or

street addresses, which will not be useful for modeling. There are also a number of protected features including gender and race, which could produce biased models.

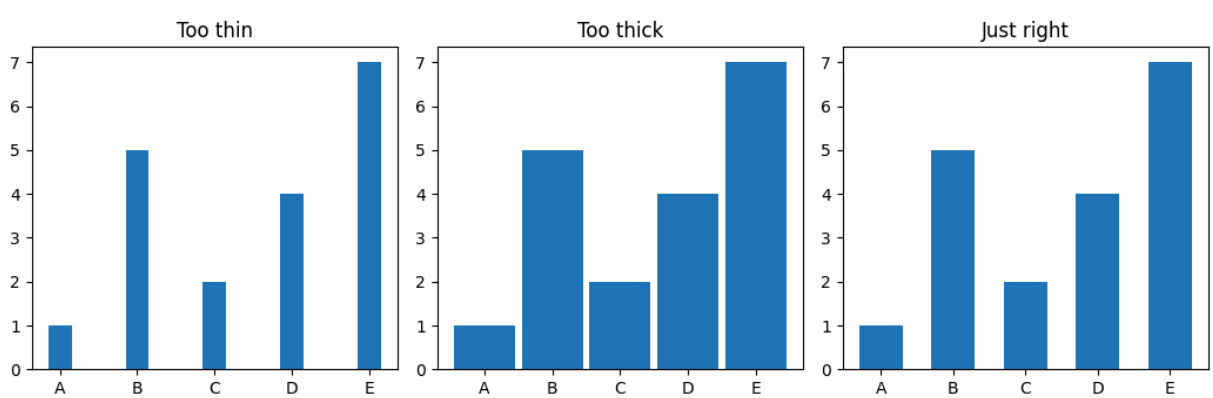
Some numeric features like the amounts and proceeds have a skewed distribution. There are a number of binary numeric features which are bimodal

Based on the boxplots, there are outliers for almost every numeric feature. This is not surprising since there are almost 1,000,000 instances in the dataset

After encoding categorical and datetime features as well as dropping features which cannot be used for modeling, we are 42 usable features.

4. Storytelling With Data graph

Reproduce any graph of your choice in p. 52-68 of the Storytelling With Data book as best you can. (The second half of chapter two). You do not have to get the exact data values right, just the overall look and feel.

In [103…

```
# Figure 2.14, page 53
categories = ['A', 'B', 'C', 'D', 'E']
heights = [1,5,2,4,7]
num_cols = 3
num_rows = 1
plt.figure(figsize=(num_cols * 3.5, num_rows * 3.5))

plt.subplot(1, 3, 1)
plt.bar(x=categories, height=heights, width=.3)
plt.title("Too thin")

plt.subplot(1, 3, 2)
plt.bar(x=categories, height=heights, width=.9)
plt.title("Too thick")

plt.subplot(1, 3, 3)
plt.bar(x=categories, height=heights, width=.6)
plt.title("Just right")

plt.tight_layout()
```